

Compressão e gerenciamento de imagens binárias

Projeto 1 - Estruturas de Dados 2

17 de setembro de 2025

1 Introdução

Nas nossas primeiras atividades, começamos o desenvolvimento de um sistema de gerenciamento de imagens que nos permitia pegar imagens em formato PGM, armazená-las em formato binário dentro de um arquivo e então processá-las através de operações de limiarização e negatização.

Vamos reaproveitar parte do que já fizemos, mas agora pensando em armazenar as imagens após a aplicação de uma operação de limiarização. Dessa forma, as imagens serão binárias no sentido de que existirão somente dois tons de cinza possíveis para cada pixel. Isso já nos permite uma redução de tamanho (como?) mesmo sem aplicação de um algoritmo de compressão, mas nos permitirá uma forma de comprimir ainda mais nossos dados.

Para além disso, vamos considerar um sistema mais próximo da realidade no qual é possível fazer remoção e compactação dos registros de imagens. Assim, teremos as seguintes funcionalidades:

- Anexação (inserção em alguma extremidade do arquivo), remoção e compactação
- Armazenamento das imagens binárias em formato comprimido dentro de um arquivo
- Um arquivo de índices para fácil localização dos registros a partir de seus *offsets*
- **Bônus:** uma operação de reconstrução da imagem original. A implementação dessa etapa vai fazer seu trabalho valer um ponto a mais (totalizando 11).

A seguir, discutimos em maior detalhe o que deve ser implementado.

2 Gerenciamento do banco de dados

Nosso arquivo contendo as imagens funcionará como base dados, que contará com um arquivo de índices para fácil acesso dos registros, que possuem tamanho variável.

A operação de remoção, como já discutimos, pode ser incluída como uma marcação no arquivo de índices, de forma que, ao ler o mesmo, um registro marcado como removido será ignorado.

Para de fato liberar espaço das imagens que foram removidas, implemente uma função de compactação que copia para um novo arquivo as entradas não removidas e então apaga o antigo. Como essa operação pode tornar-se muito custosa para bancos de dados grandes, faça-a de forma que tenha complexidade temporal linear no número de registros e complexidade de armazenamento constante.

Por fim, implemente também uma função que pegue uma imagem do banco de dados e salve-a em formato PGM.

3 Compressão

As imagens salvas na base de dados estarão em um formato comprimido. Vamos implementar um algoritmo simples que se aproveita do caráter binário da imagem e utiliza um conceito visto em aula, que é o de comprimir sequências de símbolos repetidos.

Como só teremos dois tons de cinza, a imagem consiste de sequências de repetições dos mesmos. Assim, não precisamos sequer de um marcador de repetição: podemos simplesmente escrever o número de vezes que um símbolo é repetido, de forma que a próxima entrada indica a repetição do outro símbolo

e assim sucessivamente. Precisamos apenas saber qual é o tom do primeiro pixel que os demais seguirão dedutivamente, então esta informação estará logo no começo do registro.

Por exemplo: para o par de símbolos $\{0, 1\}$ uma sequência $[0, 5, 3, 1, 4]$ é a forma comprimida de 0000011101111. No caso de imagens, essa compressão se dá sobre a matriz de pixels, já que as informações de formato, tamanho e número de cores devem continuar armazenadas para posterior recuperação da imagem.

Assim, você deve implementar uma função de *encoding* que comprime a matriz de pixels para salvá-la na base de dados. Quando o usuário requisita a imagem, seu registro é localizado na base de dados e então uma função de *decoding* descomprime a imagem para posterior processamento desta.

A binarização da imagem se dá através da aplicação de limiarização da mesma. Assim, diferentes imagens como o mesmo nome podem ser salvas e devem ser diferenciadas pelo processo de limiarização aplicado a cada uma destas. Logo, quando o usuário deseja acessar uma imagem, precisa especificar se nome e o valor de limiar que foi utilizado na sua compressão.

4 Reconstrução

Essa é uma parte bônus do trabalho, para torná-lo mais divertido. Se você implementar, sua nota máxima possível será 11.

Como podemos ter diferentes versões da mesma imagem no banco de dados (cada uma correspondente a um valor de limiar), é justo assumir que o conjunto destas imagens nos permitirá de alguma forma recuperar a imagem original. De fato, a recuperação total a partir dessas imagens binárias não é uma tarefa simples (não saberia nem garantir se isso é possível). No entanto, vamos ver o quão ruim é uma abordagem simples.

Aproveitando-se da existência de múltiplas versões binárias de uma mesma imagem, faça uma funcionalidade que retorne a imagem média dessas versões. Ou seja, some as matrizes de pixels (após *decoding*), divida o resultado pelo número de versões encontradas e então imprima o resultado para um arquivo PGM.

4.1 Como a reconstrução pode ser mais robusta?

Aqui fica uma breve discussão sobre como poderíamos reconstruir a imagem original de forma mais correta. É apenas uma curiosidade e sua implementação não faz parte do trabalho.

Ao pesquisar por uma imagem para reconstrução, encontramos um conjunto de n versões binárias da mesma, que denotaremos por $\{I^{(k)}\}_{k=1}^n$. Para cada uma destas imagens, atribuiremos um coeficiente α_k e faremos a reconstrução I_R a partir de uma soma ponderada:

$$I^R = \sum_{k=1}^n \alpha_k I^{(k)}. \quad (1)$$

O que você implementará corresponde ao caso em que $\alpha_k = 1/n$. Uma forma mais robusta seria *aprender* coeficientes que gerem uma reconstrução mais próxima da imagem real. Sendo I^O a imagem original, nosso erro de reconstrução corresponde ao quadrado da diferença pixel a pixel entre esta e sua reconstrução:

$$E = \sum_{i,j} (I_{i,j}^R - I_{i,j}^O)^2. \quad (2)$$

A partir de um **algoritmo de otimização** pode-se encontrar um conjunto de coeficientes que minimizem E , gerando assim a melhor reconstrução possível segundo esta métrica.